

Exploiting Asymmetric SIMD Register Configurations in ARM-to-x86 Dynamic Binary Translation

Yu-Ping Liu*, Ding-Yong Hong[†], Jan-Jan Wu[†], Sheng-Yu Fu*, Wei-Chung Hsu*

*Department of Computer Science and Information Engineering, National Taiwan University

{r04922005, d03922013, hsuwc}@csie.ntu.edu.tw

[†]Institute of Information Science, Academia Sinica

{dyhong, wuj}@iis.sinica.edu.tw

Abstract—Processor manufacturers have adopted SIMD for decades because of its superior performance and power efficiency. The configurations of SIMD registers (i.e., the number and width) have evolved and diverged rapidly through various ISA extensions on different architectures. However, migrating legacy or proprietary applications optimized for one guest ISA to another host ISA that has fewer but longer SIMD registers through binary translation raises the issues of asymmetric SIMD register configurations. To date, these issues have been overlooked. As a result, only a small fraction of the potential performance gain is realized due to underutilization of the host's SIMD parallelism and register capacity.

In this paper, we present a novel dynamic binary translation technique called spill-aware SLP (saSLP), which combines short ARMv8 NEON instructions and registers in the guest binary loops to fully utilize the x86 AVX host's parallelism as well as minimize register spilling. Our experiment results show that saSLP improves the performance by 1.6X (2.3X) across a number of benchmarks, and reduces spilling by 97% (99%) for ARMv8 NEON to x86 AVX2 (AVX-512) translation.

Keywords—Dynamic binary translation; SIMD; SLP

I. INTRODUCTION

Dynamic binary translation (DBT) is a widely used approach that realizes cross-ISA migration by emulating the application binaries of one guest ISA on a host machine with a different ISA. The advantage of not requiring source recompilation makes the use of cross-ISA DBTs critical in migrating legacy or proprietary applications from one ISA to another. For example, many Android applications include ARM native code for performance critical parts. With the DBT technique, the widely used Android emulators can run such ARM binaries on x86 platforms for application streaming [1], development [2], [3], and security analysis [4], [5]. As the efficiency of the emulation process is crucial to the success of cross-ISA DBTs, optimizing the translated code is a key issue in the design of cross-ISA DBTs.

Single instruction multiple data (SIMD) instruction sets provide both execution and power efficiency by exploiting fine-grained data parallelism in a wide range of applications [6]–[8]. As a result, all major processor manufacturers have adopted them as extensions of general purpose ISAs; for example, NEON for ARM, SSE/AVX for x86 and AltiVec for PowerPC. Moreover, SIMD architectures have been

improved through the introduction of additional features and extra wider registers. For instance, the Intel x86 architecture has evolved from SSE, which has 16 16-byte registers with elementary vector operations, to the current AVX-512 where 32 64-byte registers can be used with more advanced instructions such as masked operations, conflict detection and gather/scatter [9]. Meanwhile, the number of 16-byte registers in the NEON extension has increased from 16 in ARMv7 to 32 in ARMv8 without extending the register width [10]. The evolution of SIMD architectures has enabled a larger number of applications to explore more parallelism. However, it has also led to divergent SIMD register configurations (i.e., the number and width) in different ISAs.

The growing importance of SIMD has reinforced the need to translate SIMD instructions efficiently in cross-ISA DBTs. A number of methods [11]–[13] have been proposed to establish feasible mappings of guest SIMD registers and instructions to the host ones. They map a guest SIMD register directly to a host register with the *same* width, and map a guest SIMD instruction to equivalent host instructions that operate on the *same* number of elements (i.e., lanes). More recent approaches [14]–[16] use loop vectorization techniques to exploit more SIMD lanes for parallelism, but still map *one* guest SIMD register to *one* host SIMD register. However, existing works fail to address the challenges raised by asymmetric SIMD register configurations in which the host ISAs may have fewer but longer SIMD registers than the guest ISAs. The above-mentioned methods either (1) underutilize the improved host parallelism; or (2) they lead to register spilling because of the excessive number of host SIMD registers required for mapping, so only a small fraction of the potential performance gain is realized.

To solve the problem, we seek to exploit longer host SIMD registers for both parallelism and register capacity. To this end, we propose a new cross-ISA DBT technique called spill-aware SLP (saSLP). Given a guest binary loop that has been optimally vectorized and unrolled according to the guest SIMD register configuration, saSLP combines multiple short guest SIMD instructions and registers to form a longer one on the host. This increases the number of elements that can be processed and stored in each host SIMD

register, and thereby improves the execution efficiency and reduces the number of host registers required for mapping.

The design of saSLP faces several challenging issues. First, combining short SIMD instructions involves a vectorization legality issue because we must ensure that there is no data dependency for reordering memory operations. This issue is more challenging for DBTs than for static compilers since DBTs work on a smaller scope (e.g., traces) and a great deal of high-level information is lost in binaries (e.g., restrict keyword in C for pointer aliasing). Hence, with limited information, DBTs may not always be able to determine the existence of data dependency at the time of translation. Second, though combining short guest SIMD registers can reduce the number of host registers used for mapping, access to an individual short register might require extra data movement (e.g., unpacking from the combined long register), which results in additional data reordering instructions on most architectures. Thus, arranging a suitable trade-off between such overheads and the benefits of reducing the number of host registers used is a critical issue. The key contributions of this paper are as follows:

- We identify the issues of asymmetric SIMD register configurations in cross-ISA DBTs. To resolve the problems, we propose saSLP, which combines short guest SIMD instructions to form a longer instruction that can be used to exploit the improved host parallelism.
- We propose a novel spill-aware cost model that can determine whether combining short guest SIMD registers would be profitable by evaluating the trade-off between the extra data reordering overheads and the benefits of reducing the number of host registers used.
- We implement saSLP in a retargetable cross-ISA DBT and evaluate it with ARMv8 NEON to x86 AVX2 (and AVX-512) translations. The benchmark results show that saSLP improves the performance by 1.6X (2.3X) and reduces register spilling by 97% (99%) on average.

The remainder of this paper is organized as follows. Section II provides an overview of cross-ISA DBTs and explains how asymmetric SIMD register configurations cause problems in cross-ISA DBTs. We describe our approach, spill-aware SLP (saSLP), in Section III and then report the evaluation results in Section IV. Section V deals with related work and Section VI contains our concluding remarks.

II. BACKGROUND AND MOTIVATION

One of the core techniques in cross-ISA DBTs is mapping guest registers and instructions to effective host ones. We begin this section with an overview of such mappings, and then explain how asymmetric SIMD register configurations cause problems of underutilization.

A. Overview of Cross-ISA Dynamic Binary Translation

Most cross-ISA DBTs translate guest binaries in the granularity of a translation code fragment and maintain a guest

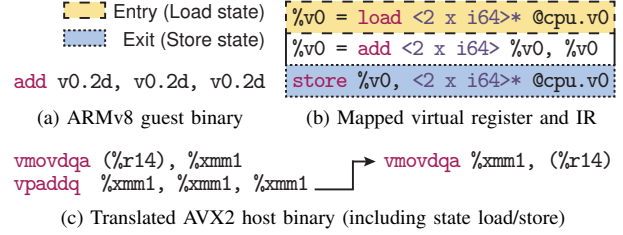


Figure 1. An example of register/instruction mapping in DBT

architectural state (i.e., virtual CPU) in the host memory for emulation. To reduce memory access, typical retargetable and cross-ISA DBTs [11], [17]–[19] map each guest SIMD register in the architectural state to a virtual register with the same width within a translation code fragment. Thus, they have to synchronize modified virtual registers back to the guest state at the exit points. Similarly, guest SIMD instructions are mapped to equivalent machine-independent IR that operates on the same number of vector elements.

For example, as illustrated in Figure 1, the 16-byte guest register `v0` in Figure 1a is mapped to virtual register `%v0` with the same width (i.e., a vector of two 8-byte integers) in Figure 1b. First, it is loaded from the guest state at the entry of the translation code fragment. It is then modified and used by the vector `add` IR instruction, and finally synchronized back to the guest state at the exit. In the final step, the DBT back-end allocates host registers to mapped virtual registers and converts mapped IR to the host binary in Figure 1c.

B. An Example of Asymmetric SIMD Register Configuration

As SIMD register configurations (i.e., the number and width) have become more asymmetric between ISAs, compilers need to consider ISA-dependent configurations when performing optimization. For example, many modern compilers, such as GCC, LLVM and Intel ICC, optimally vectorize and unroll loops according to the SIMD register configurations of target ISAs to exploit both data and instruction level parallelism (ILP) [20]–[22]. The example in Figure 2 illustrates the procedure. For ease of presentation, we scale down the number of SIMD registers in ARMv8 NEON and x86 AVX2 from 32 and 16 to 6 and 3 respectively.

Next, we explain how modern compilers optimize the loop in Figure 2a for the ARMv8 NEON target. First, the compiler vectorizes the loop by maximizing the number of elements processed simultaneously in a SIMD register. For example, in Figure 2b, each of the `v0-v4` registers holds four 4-byte floating-point numbers. Second, to achieve better ILP, the compiler unrolls and interleaves the loop according to the number of available SIMD registers, which is six in this example. The loop is only unrolled once because a larger unrolling factor would require more than six registers and cause register spilling. The final compiled binary is shown in Figure 2b. NEON instructions from the first and the second unrolled iteration are marked ①–⑤ and ①–⑤ respectively.

```

void daxpy(int n, float da, float *dx,
           float *dy, ...)
{
    ...
    for (int i = 0; i < n; ++i)
        dy[i] += dx[i] * da;
}

```

(a) C source code

```

L0: ldr v0, [x1] ; dx[i:i+4] ①
    ldr v1, [x2] ; dy[i:i+4] ②
    ldr v3, [x2, #16] ③
    fmul v0.4s, v0.4s, v4.4s ④
    fadd v1.4s, v1.4s, v0.4s ⑤
    str v1, [x2] ⑥
    ldr v2, [x1, #16] ⑦
    add x1, x1, #32
    cmp x1, x0
    fmul v2.4s, v2.4s, v4.4s ③
    fadd v3.4s, v3.4s, v2.4s ④
    str v3, [x2, #16] ⑤
    add x2, x2, #32
    b.lt L0

```

(b) ARMv8 NEON guest binary (vectorized and unrolled)

```

Entry: load %x0 %x1 %x2 %v4
Loop: ① %v0 = load <4 x f32>* %x1
      ② %v1 = load <4 x f32>* %x2
      ③ %v3 = load <4 x f32>* [%x2 + 16]
      ④ %v0 = fmul <4 x f32> %v0, %v4
      ⑤ %v1 = fadd <4 x f32> %v1, %v0
      ⑥ store %v1, <4 x f32>* %x2
      ⑦ %v2 = load <4 x f32>* [%x1 + 16]
      %x1 = add i64 %x1, 32
      %cc = icmp ult i64 %x1, %x0
      ⑧ %v2 = fmul <4 x f32> %v2, %v4
      ⑨ %v3 = fadd <4 x f32> %v3, %v2
      ⑩ store %v3, <4 x f32>* [%x2 + 16]
      %x2 = add i64 %x2, 32
      br i1 %cc, label %Loop, %Exit
Exit: store %x1 %x2 %v0 %v1 %v2 %v3 %cc

```

(c) Mapped virtual registers and IR with typical methods

```

Entry: load %x0 %x1 %x2 %v4
      %v44 = pack2 <4 x f32> %v4, %v4
Loop: ② ② %v13 = load <8 x f32>* %x2
      ① ① %v02 = load <8 x f32>* %x1
      %x1 = add i64 %x1, 32
      %cc = icmp ult i64 %x1, %x0
      ③ ③ %v02 = fmul <8 x f32> %v02, %v44
      ④ ④ %v13 = fadd <8 x f32> %v13, %v02
      ⑤ ⑤ store %v13, <8 x f32>* %x2
      %x2 = add i64 %x2, 32
      br i1 %cc, label %Loop, %Exit
Exit: %v0, %v2 = unpack2 <8 x f32> %v02
      %v1, %v3 = unpack2 <8 x f32> %v13
      store %x1 %x2 %v0 %v1 %v2 %v3 %cc

```

(d) Transformation result of saSLP for x86 AVX2 host

Figure 2. Limitations of typical mapping methods and the saSLP solution for asymmetric SIMD register configuration

For cross-ISA DBTs, such ISA-dependent optimizations in compilers can cause problems in the typical register and instruction mapping methods discussed earlier. As the SIMD register configurations are asymmetric between the guest and host ISAs, a guest binary loop that has been optimized according to the guest configuration may become non-optimal on the host after typical mapping. For instance, let us consider how typical mapping methods translate the ARMv8 NEON guest binary in Figure 2b to the x86 AVX2 host. As shown in Figure 2c, typical methods map each 16-byte NEON register (v0-v4) to one virtual register (%v0-%v4) with the same width, resulting in a total of five live ranges. All NEON instructions are mapped to IR instructions that also operate on four 4-byte floating-point numbers. As a result, the parallelism that can operate on eight 4-byte floating-point numbers simultaneously on the AVX2 host is underutilized. Furthermore, since the AVX2 host in this case has only three registers, the five overlapping live ranges in Figure 2c will cause register spilling in the resulting host binary, and only half of the 32-byte host register capacity can be used to store a 16-byte guest register.

To solve these problems, the proposed saSLP algorithm transforms the IR in Figure 2c to that in Figure 2d. First, it combines each of the five instructions ①-⑤ in the first loop iteration with the five instructions ①-⑤ in the second unrolled iteration in a pairwise manner. Therefore, the combined instructions (e.g., ③ ③) in Figure 2d can now operate on eight 4-byte floating-point numbers simultaneously, and thus fully utilize the AVX2 parallelism. Second, saSLP combines four 16-byte virtual registers (%v0-%v3) in Figure 2c

to form two 32-byte virtual registers (%v02 and %v13) in Figure 2d. As a result, register spilling will no longer occur because the number of overlapping live ranges is reduced to three by fully utilizing the AVX2 register capacity.

However, saSLP faces several challenges when performing the above transformation. First, to combine the instructions in Figure 2c, saSLP must ensure there is no data dependency between the reordered memory instructions. This is more challenging in DBTs compared to static compilers since DBTs translate code fragments with small granularity. With the limited information that can be derived from a narrow scope, saSLP may not always be able to determine whether the reordering would violate the dependency at the translation time. For example, the order of load ① and store ⑤ in Figure 2c is swapped in Figure 2d, and they access the memory locations %x1 + 16 and %x2 respectively. Since the pointer values, %x1 and %x2 are unknown at the time of translation, a conditional translation approach with a runtime check is required to avoid the execution of transformed code in Figure 2d when %x1 and %x2 are aliased.

Second, combining short registers in Figure 2c can reduce the number of live ranges and thus saves host registers. However, to access an individual combined short register, it is necessary to unpack the short register from the corresponding long register. For example, the two modified long registers (%v02 and %v13) in Figure 2d need to be unpacked to four short registers (%v0-%v3) and then stored back in the guest state for synchronization. The overheads of extra data reordering might nullify the benefits of saving host registers. Thus, a cost model that can precisely estimate the gain of

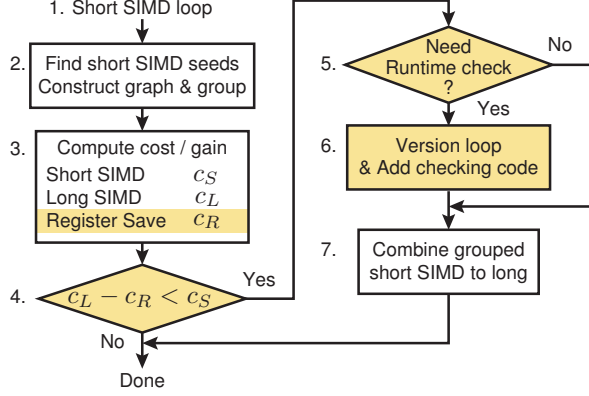


Figure 3. Overview of the saSLP algorithm

saving host registers and the cost of extra data reordering is required to determine if short registers should be combined.

III. SPILL-AWARE SLP

In this section, we present our approaches for the above-mentioned transformation, and then provide details of the solutions to the two challenges in Section III-C and Section III-D respectively. Spill-aware SLP (saSLP) is an optimization algorithm designed for cross-ISA DBTs. First, we introduce the algorithm and then describe how the instructions are combined and how the registers are combined.

A. Overview

Figure 3 details the steps of the saSLP algorithm. The white boxes are related to combining short SIMD instructions to form longer ones; and the highlighted parts are designed to address the two challenges mentioned earlier.

As shown in Figure 3, saSLP targets translation code fragments that contain short SIMD loops. They are good optimization candidates because SIMD loops are usually the hot innermost loops (step 1). Then saSLP collects consecutive short SIMD stores in the loop body as seeds. Starting with those seeds, saSLP constructs a combining graph by traversing the use-define chains, and then groups visited short SIMD instructions that cannot be combined to form a long SIMD instruction. The grouped short SIMD instructions in each group are candidates for being combined with each other to form a long SIMD instruction (step 2).

For each constructed combining graph, saSLP estimates the performance cost of the short and the long SIMD forms (i.e., c_S , c_L in step 3). The long SIMD cost also includes the extra overheads of data movement (e.g., packing and unpacking). Then, saSLP credits the additional gain of saving SIMD registers (i.e., c_R in step 3). If combining short SIMD instructions is profitable (step 4), saSLP further determines if a runtime memory check is required to detect memory dependency, which cannot be determined at the translation time (step 5). If a runtime check is required, saSLP versions the

Algorithm 1: Construct Combining Graph

input : A slice of seed instructions S .
output: A graph G for combining grouped instructions.

```

1 Function construct_graph( $S$ )
2   Initialize  $G$  as an empty graph.
3   traverse_and_construct( $S$ )
4   return  $G$ 
5 Function traverse_and_construct( $S_1$ )
6   if  $S_1$  can be combined to form long SIMD then
7      $G.add\_group(S_1)$ 
8     foreach  $S_2 \in$  operand lists of  $S_1$  do
9       if  $S_2$  is a list of short SIMD then
10        traverse_and_construct( $S_2$ )
11         $G.add\_edges(S_1, S_2)$ 
12   else
13      $G.add\_node\_and\_pack(S_1)$ 

```

loop into an original short SIMD version and an optimized long SIMD version, and produces runtime checking code to decide which version to execute at runtime (step 6). Finally, saSLP combines the grouped short SIMD instructions to form longer SIMD instructions in the long SIMD version loop, and inserts additional data reordering instructions for packing and unpacking (step 7).

We use the simplified case of ARMv8 NEON to x86 AVX2 translation in Figure 2 as a running example. We use the terms *ShortVS* and *LongVS* to denote, respectively, the SIMD vector width in the input IR (guest) and the output IR (host). In addition, *CF* (i.e., combining factor) denotes how many short instructions will be combined to form a long instruction. *CF* is computed as $LongVS / ShortVS$.

B. Combining Short SIMD to Form Long SIMD

The input of the saSLP algorithm is IR of a short SIMD loop, which is mapped from the guest binary by the typical mapping methods described in Section II-B. To find the starting points to construct combining graphs, saSLP first collects all consecutive short SIMD store instructions in the loop body to form seed chains. Selecting consecutive stores as seeds is a popular heuristic approach that many compilers adopt [23], [24]. This is because adjacent memory stores usually have a higher probability of storing data from a sequence of similar instructions. In the motivating example, where the *ShortVS* is 16 bytes (Figure 4a), saSLP collects the two consecutive 16-byte SIMD stores (⑤ and ⑥), which access, respectively, the memory locations $\%x2$ and $\%x2 + 16$ to form a seed chain of length two.

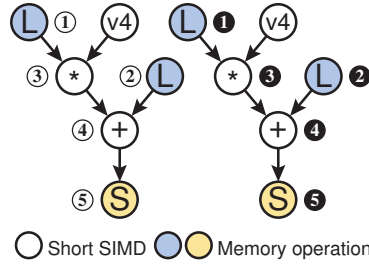
For each seed chain comprised of several consecutive short SIMD stores, saSLP slices the chain with the size of CF consecutive stores (i.e., the width of each slice is $CF \times ShortVS = LongVS$) and uses the graph construction algorithm to construct a combining graph starting from each slice. As shown in Algorithm 1, the combining graphs are

```

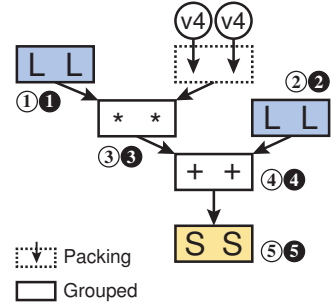
Loop: ① %v0 = load <4 x f32>* %x1
      ② %v1 = load <4 x f32>* %x2
      ③ %v3 = load <4 x f32>* [%x2 + 16]
      ④ %v0 = fmul <4 x f32> %v0, %v4
      ⑤ %v1 = fadd <4 x f32> %v1, %v0
      ⑥ store %v1, <4 x f32>* %x2
      ⑦ %v2 = load <4 x f32>* [%x1 + 16]
      %x1 = add i64 %x1, 32
      %cc = icmp ult i64 %x1, %x0
      ⑧ %v2 = fmul <4 x f32> %v2, %v4
      ⑨ %v3 = fadd <4 x f32> %v3, %v2
      ⑩ store %v3, <4 x f32>* [%x2 + 16]
      %x2 = add i64 %x2, 32
      br i1 %cc, label %Loop, %Exit

```

(a) Input IR of the saSLP algorithm



(b) Data dependency graph



(c) Constructed graph and groups

Figure 4. An example of saSLP combining graph

constructed in a bottom-up manner from the seeds with recursive traversal (line 3 and 10) along the use-define chains (i.e., edges in Figure 4b). For a list of short SIMD instructions S_1 currently visited by the traversal, the algorithm first checks whether the CF short SIMD instructions in S_1 can be combined to form a long SIMD instruction (line 6).

Short SIMD instructions in S_1 that can be combined must satisfy all of the following requirements: (1) have the same opcode, data type and reside in the same basic block; (2) have not been inserted into the combining graph G ; (3) do not have a duplicated instruction in S_1 ; (4) if S_1 is a list of loads, they must be consecutive; and (5) they can be scheduled in parallel. The last requirement ensures that all data flow and memory dependency are preserved after combining the short SIMD instructions. The reason is that the combined short SIMD instructions are moved to the position of the resulting long SIMD instruction. Therefore, to combine short SIMD instructions that involve the reordering of memory instructions, saSLP needs to check whether reordered memory instructions use pointers that may alias with each other. Static compilers [24] usually do this via intra- or inter-procedural alias analysis. However, as explained earlier, DBTs have difficulty determining such aliasing at the translation time. To solve this problem, reordered memory instructions with pointers that can be checked at runtime with techniques introduced in Section III-C are marked as combinable. Others are treated as non-combinable.

If all CF short SIMD instructions in S_1 can be combined, they are grouped and inserted into the combining graph G (line 7 and solid-line rectangles in Figure 4c). Then, the traversal continues to trace along the use-define chains from S_1 to S_2 (i.e., following the arrows in Figure 4b in reverse order), where S_2 is a list of short SIMD operands of the instructions in S_1 (line 8-9). Finally, after the recursive traversal (line 10) returns, edges connecting the newly inserted nodes of S_1 and S_2 are added to the combining graph G (line 11 and arrows in Figure 4c).

For short SIMD instructions in S_1 that cannot be combined, the graph construction algorithm inserts a short SIMD node for each short SIMD instruction in S_1 into the com-

binning graph G and adds an extra packing operation node to pack the short SIMD nodes into a long SIMD node (line 13 and the dashed-line rectangle in Figure 4c). Therefore, the resulting long SIMD node can be used by other long SIMD nodes because they now have the same width. The traversal returns to the previous level at non-combinable S_1 .

In the motivating example where CF is 2 (i.e., 32/16 because $ShortVS = 16$ and $LongVS = 32$ in ARMv8 NEON to x86 AVX2 translation) as shown in Figure 4a, saSLP first slices the collected seed chain into the size of two (CF) 16-byte stores, and then uses Algorithm 1 to construct a combining graph for each slice. The graph construction starts the traversal from a seed slice, which is a list of two consecutive 16-byte stores ($S_1 = \langle \textcircled{5}, \textcircled{5} \rangle$), and checks whether $\textcircled{5}$ and $\textcircled{5}$ can be combined to form a long 32-byte store. Obviously, $\textcircled{5}$ and $\textcircled{5}$ satisfy requirements (1)-(4), but further analysis is needed to determine if they can be scheduled in parallel. As combining $\textcircled{5}$ and $\textcircled{5}$ needs to reorder memory instructions $\textcircled{5}$ and $\textcircled{1}$, which respectively store and load 16-byte data at memory locations $\%x2$ and $\%x1 + 16$, the algorithm needs to ensure that $\%x2$ and $\%x1$ are not aliased to schedule $\textcircled{5}$ and $\textcircled{5}$ in parallel. Therefore, the graph construction algorithm records the pair of pointers $\langle \%x2, \%x1 + 16 \rangle$ that can be checked at runtime and marks $\langle \textcircled{5}, \textcircled{5} \rangle$ as combinable.

Since $\textcircled{5}$ and $\textcircled{5}$ can be combined to form a long SIMD instruction, they are grouped as $\textcircled{55}$ and inserted into the combining graph G , as shown in Figure 4c. Then, the traversal follows the use-define chains to visit the add $\langle \textcircled{4}, \textcircled{4} \rangle$, multiply, $\langle \textcircled{3}, \textcircled{3} \rangle$ and load instruction $\langle \textcircled{1}, \textcircled{1} \rangle$ lists. All of them are grouped and inserted into the combining graph G . When the traversal visits the list of right-hand side operands of the multiply instructions $\langle \textcircled{3}, \textcircled{3} \rangle$, requirement (3) is violated because both $\textcircled{3}$ and $\textcircled{3}$ use short SIMD register $\%v4$ as their right-hand side operands. As a result, two short SIMD nodes of $\%v4$ are inserted into G with an additional packing operation node, as shown in Figure 4c.

Finally, the last list of load instructions $\langle \textcircled{2}, \textcircled{2} \rangle$ is grouped and inserted into the graph G , thereby completing the construction of the combining graph. The final graph is shown in Figure 4c, where the grouped short SIMD instructions are

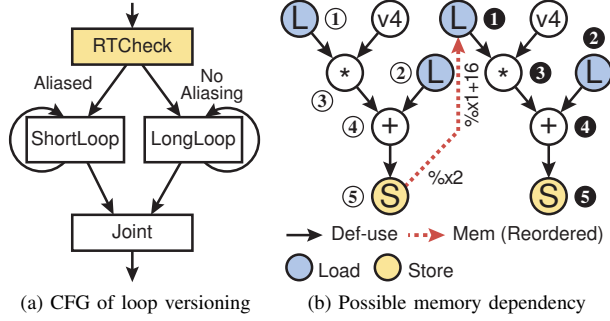


Figure 5. Overview of runtime check for possible dependency

ready to be combined to form long SIMD instructions.

C. Runtime Memory Aliasing Check

Recall that saSLP must ensure that there is no data dependency between the reordered memory instructions. However, due to the limitations of DBTs, the data dependency caused by pointer aliasing may not always be determined at the translation time. To solve this problem, we propose a conditional translation with runtime check of pointer aliasing.

When saSLP encounters memory dependency that cannot be determined at the translation time, it versions the input loop into an original short SIMD loop (ShortLoop) and an optimized long SIMD loop (LongLoop), and inserts checking code into the RTCheck block. This operation predicates the two versioned loops based on whether aliased pointers are detected at runtime (Figure 5a). Recall that pointer pairs that require runtime check are collected when the combining graph is constructed. Given a pair of may-alias pointers $\langle p, q \rangle$, saSLP first checks if (1) the two pointers start from loop invariant addresses (*StartAddr*) and move with constant strides (*StepSize*) in each loop iteration; and (2) the input loop has a loop invariant trip count (*TripCnt*). These requirements ensure that saSLP can derive two loop invariant boundaries (i.e., $[StartAddr, StartAddr + TripCnt \times StepSize]$ interval) to respectively bound the memory region accessed by p and q during the entire loop. Then, whether or not p and q are aliased can be verified at runtime by checking if their regions overlap. Any pair of overlapped regions will force the execution to jump to the ShortLoop for legality. If $\langle p, q \rangle$ does not meet requirements (1) and (2), they are treated as aliased during the construction of combining graphs. However, since saSLP targets short SIMD loops, which tend to be regular, these requirements are usually satisfied.

We now consider the two memory instructions ⑤ and ①, which respectively store and load 16-byte data at memory location $p = \%x2$ and $q = \%x1 + 16$, as shown in Figure 4a. Because the values of $\%x1$ and $\%x2$ are unknown at the translation time, there may be data dependency between ⑤ and ①, as shown in Figure 5b. To verify that such a dependency does not exist at runtime, saSLP analyzes the loop IR in Figure 4a and concludes that (1) both $\%x1$ and $\%x2$ start

with loop invariant values (i.e. $\%x1.init$ and $\%x2.init$) loaded from the guest state at the entry of the translation code fragment and move with a 32-byte constant stride; and (2) the loop has a loop invariant trip count $\lceil (\%x0 - \%x1.init)/32 \rceil$. Therefore, saSLP inserts checking code into RTCheck. The code evaluates the accessed regions of $\%x2$ and $\%x1 + 16$ and determines if the two regions overlap at runtime.

The runtime check is further optimized by grouping pointers with constant distances and the same stride (e.g., the pointers of ⑤ and ⑤) and performing the overlap tests between pointer groups instead of between individual pointers. Since saSLP targets optimally vectorized and unrolled guest SIMD loops where many pointers can be grouped, this optimization reduces the number of overlap tests significantly.

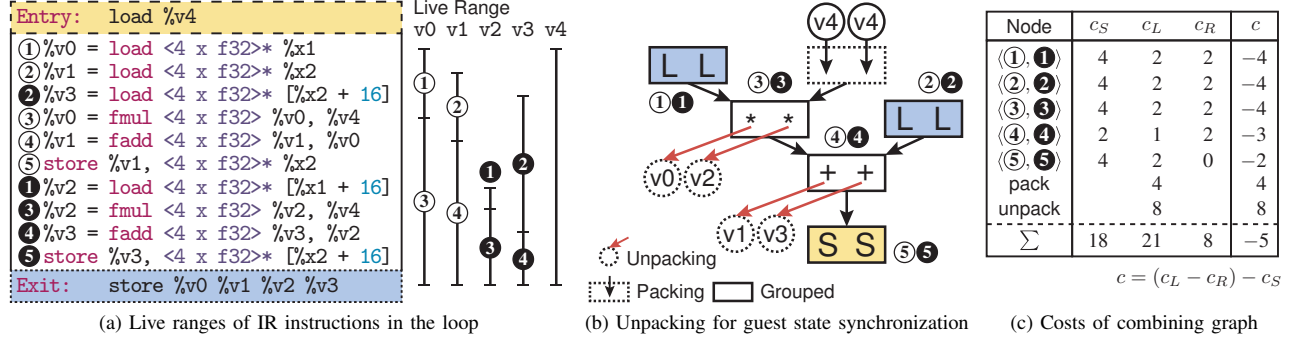
D. Spill-aware Cost Model

To decide whether combining the grouped short SIMD instructions in the combining graph to form long SIMD instructions would be profitable, we devised a new spill-aware cost model to evaluate the overall performance cost.

To model the performance cost, the spill-aware cost model first categorizes cost factors into three classes: (1) the short SIMD cost c_S , which denotes the cost of executing SIMD instructions with *ShortVS* width; (2) the long SIMD cost c_L , which denotes the cost of executing combined SIMD instructions with *LongVS* width as well as the data reordering overheads of packing and unpacking between short and long SIMD registers; and (3) the register saving gain c_R , which is the benefit of eliminating register spilling by combining multiple short SIMD registers to form a longer register. For each node N_i in the combining graph G , the cost model computes $c_S(N_i)$, $c_L(N_i)$ and $c_R(N_i)$, and then determines if combining the grouped short SIMD instructions in G will be profitable if

$$\sum_{N_i \in G} (c_L(N_i) - c_R(N_i)) < \sum_{N_i \in G} c_S(N_i) \quad (1)$$

Since compilers [24] usually provide an interface to estimate the cost of executing a specific IR instruction on the target processor (e.g., the latency of corresponding target instruction), our cost model largely reuses the existing interfaces with minor modifications to compute c_S and c_L . Therefore, we focus on the computation of c_R . The core idea of c_R is assigning higher c_R to short SIMD instructions with live ranges that are likely lead to register spilling. Thus, such live ranges can have a greater chance of being combined and stored in long SIMD registers to avoid spilling. First, we define the following terms for a SIMD instruction n_i . (1) $Reg(n_i)$ denotes the physical SIMD register capacity occupied by the output of n_i ; (2) $Int(n_i)$ denotes the maximal physical SIMD register capacity occupied by the live ranges that interfere with the live range of n_i at the same time; (3) $Avail(n_i)$ denotes the total physical SIMD register capacity that can be used to store the output of n_i . Hence,



if $Int(n_i) \geq Avail(n_i)$, then n_i will lead to register spilling because its live range will interfere with too many live ranges at the same time and thus overwhelm the available SIMD register capacity on the host.

Given a list of CF grouped short SIMD instructions $S_i = \langle s_1, \dots, s_{CF} \rangle$ that can be combined to form a long SIMD instruction l_i , the cost model computes the register saving gain $c_R(S_i)$ as follows:

$$\frac{1}{ShortVS} \sum_{s_j \in S_i} \begin{cases} Reg(s_j) - \frac{Reg(l_i)}{CF} & \text{if } Int(s_j) \geq Avail(s_j) \\ 0 & \text{, otherwise} \end{cases} \quad (2)$$

In Equation 2, the output of each short SIMD instruction s_j respectively occupies $Reg(s_j)$ bytes and $Reg(l_i) / CF$ bytes (amortized over S_i) before and after the combining operation. Therefore, $Reg(s_j) - Reg(l_i) / CF$ is the register capacity that can be saved by combining s_j . If s_j could lead to register spilling (i.e., $Int(s_j) \geq Avail(s_j)$), the cost model rewards s_j with the saving in the unit of $ShortVS$.

To compute the register saving gain c_R , the cost model performs the following procedure. First, it derives $Reg(s_i)$, $Int(s_i)$ and $Avail(s_i)$ for each short SIMD instruction s_i in the combining graph. Second, for $Reg(s_i)$ and $Avail(s_i)$, it maps the data type of s_i to the physical register class on the host, and uses the width and total capacity of mapped physical register class as $Reg(s_i)$ and $Avail(s_i)$. Finally, it computes the live range of each short SIMD instruction in the loop with typical liveness analysis [25] and then derives $Int(s_i)$ for each short SIMD instruction s_i .

Next, we consider the same motivating example where the AVX2 host has three 32-byte YMM registers. The cost model computes the costs for each short SIMD instruction s_i based on their data types and live ranges shown in Figure 6a. For instance, the grouped short SIMD multiply instructions ③ and ③ have two live ranges before being combined. Both of them have a 16-byte data type. Since the 16-byte upper half of a 32-byte YMM register cannot be used when the 16-byte lower half (i.e. XMM sub-register) is read or written by SIMD instructions operating on XMM, ③ and ③ occupy the entire 32-byte capacity of the YMM registers

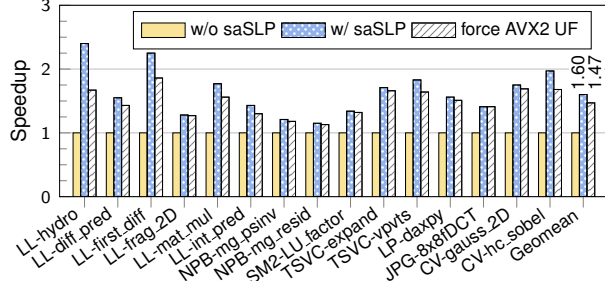
($Reg(\textcircled{3}) = Reg(\textcircled{3}) = 32$), but only operate on 16-bytes of data. After combining, the resulting long SIMD instruction ③③ only occupies a YMM register ($Reg(\textcircled{3}\textcircled{3}) = 32$).

Moreover, the live range of ③ interferes with a maximum of four 16-byte live ranges (but each occupies a 32-byte YMM) at the same time, as shown in Figure 6a. Hence, ③ could lead to register spilling since $Int(\textcircled{3}) = 4 \times 32 \geq Avail(\textcircled{3}) = 3 \times 32$. This also holds for ③. Consequently, as shown in Equation 2, combining $\langle \textcircled{3}, \textcircled{3} \rangle$ with $CF = 2$ could save $2 \times (32 - 32/2) = 32$ bytes of register capacity, which is equal to two $ShortVS$ and implies that $c_R(\langle \textcircled{3}, \textcircled{3} \rangle) = 2$.

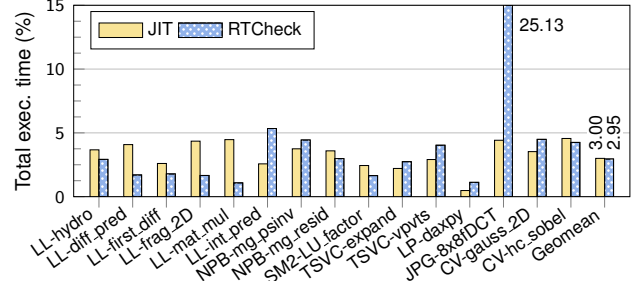
To compute c_S and c_L , the cost model queries the back-end interface of our DBT and finds that the cost of SIMD multiply instructions is 2 regardless of whether it operates on four or eight 4-byte floating-point numbers on the AVX2 host. Therefore, as shown in Figure 6c, the cost model concludes that $c_S(\langle \textcircled{3}, \textcircled{3} \rangle) = 2 + 2$ and $c_L(\langle \textcircled{3}, \textcircled{3} \rangle) = 2$. Moreover, recall that all modified registers need to be synchronized back to the guest state. Thus, two extra unpacking instructions are required to respectively extract $\%v0$ and $\%v2$ from the long SIMD instruction ③③, as shown in Figure 6b. The additional cost of extracting a 16-byte SIMD register from a 32-byte SIMD register is 2 based on the back-end interface. Therefore, unpacking ③③ adds an extra cost of 4 to the unpack row in Figure 6c. The unpacking of ④④ also incurs an extra cost of 4. The cost model repeats the above computation on each node in the combining graph and produces the final cost table shown in Figure 6c. As the performance cost of the long SIMD form ($c_L - c_R = 13$) is lower than the short SIMD one ($c_S = 18$), combining the grouped short SIMD instructions is profitable.

IV. EVALUATION

We implemented saSLP in HQEMU [11], a retargetable cross-ISA DBT based on the LLVM 3.9 JIT compiler. The guest binary is first translated to LLVM IR and then transformed by saSLP. The LLVM back-end compiles the transformed IR and produces the host binary. We evaluated saSLP with translations from a guest ISA, ARMv8 NEON with 32 16-byte SIMD registers, to two host ISAs, x86 AVX2 and



(a) Performance of NEON to AVX2 translation



(b) Overheads of JIT and runtime aliasing checks

Figure 7. Performance results with AVX2 (AVX-256) host

AVX512. AVX2 and AVX-512 have, respectively, 16 32-byte and 32 64-byte SIMD registers. Fifteen benchmarks, in which SIMD execution occupies over 40% of the total execution time, were selected from various benchmark suites for the evaluation: Livermore Loops (LL) [26], NAS Parallel Bench (NPB) [27], SciMark 2 (SM2) [28], Test Suite for Vectorizing Compilers (TSVC) [29], LINPACK (LP) [30], cjpeg (JPG) [31] and OpenCV (CV) [32]. All benchmarks were compiled using AArch64 GCC 5.4.0 with flags “-O3 -ftree-vectorize -funroll-loops -ffast-math -static” to let the guest compiler automatically vectorize and unroll loops according to the 32 available 16-byte NEON registers. For comparison, we used the translation without saSLP as our performance baseline, where NEON instructions were translated to AVX-128 instructions which have the same SIMD width. The host systems for AVX2 and AVX-512 translation are, respectively, Intel Core i7-6700 (Skylake) at 3.4GHz with 64GB of RAM and Intel Xeon Phi 7210 (Knights Landing) at 1.3GHz with 109GB of RAM.

A. Performance Results with AVX2 (AVX-256)

Figure 7a shows the speedup of NEON to AVX2 translation with saSLP. By combining two NEON instructions to form an AVX2 instruction, saSLP achieves a speedup of 1.6X on average compared to that without saSLP. Theoretically, in this experiment we could achieve a maximum speedup of 2X by the doubled parallelism factor from AVX2. However, the actual improvement that can be accomplished in practice is affected by the following factors. (1) Cache effects and the working set size: if the working data cannot fit the L1 data cache, it is difficult to maintain the theoretical SIMD computation throughput due to the latency of L1 cache miss. (2) Memory access patterns: SIMD loads/stores usually require more restrictions on the access patterns (e.g., alignment); thus, loops with imperfect access patterns can incur an additional overhead because of extra data fetching and reassembling [33]. (3) According to Amdahl’s law, the speedup is limited by the ratio of non-SIMD code.

Most of the benchmarks achieve more than 40% performance improvement with saSLP, as shown in Figure 7a. In particular, the benchmarks *LL-hydro*, *LL-first_diff*, *LL-mat_mul*, *TSVC-vpvt* and *CV-hc_sobel* achieve significant

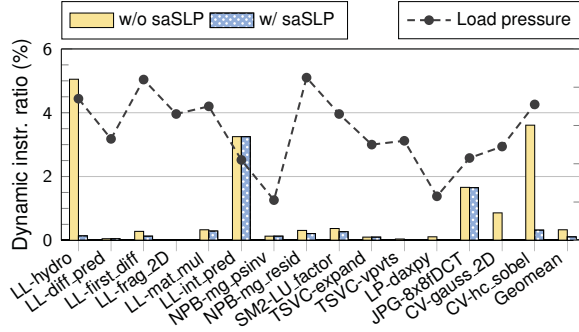
speedups over 1.8X. This is because their SIMD loops have good access patterns and locality, and contain a high ratio of SIMD computation. In contrast, *NPB-mg_psinv* and *NPB-mg_resid* achieve an average speedup of around 1.2X, which is relatively low. We used Intel VTune Amplifier to profile these two benchmarks. The results indicate that *NPB-mg_psinv* and *NPB-mg_resid* have 17% of the total cycles with split loads (i.e., fetching two cache lines because the load crosses the 64-byte cache line boundary), which degrades the performance [33]. Moreover, the working set of these two benchmarks is larger than the L3 cache size, leading to 30% cycles on L3 miss stalls. Since these memory overheads cannot be reduced by the improved parallelism, there is little room for saSLP to enhance the performance.

The overheads of JIT compilation (including optimization) and runtime aliasing checks are measured as the ratios to the total execution time in Figure 7b. Since all kernel loops in the benchmark set are translated once but iterate many times, the JIT overhead is amortized to 3% on average. Moreover, our DBT uses another thread for LLVM JIT so that the overhead can often be hidden. The measurement also shows the runtime aliasing checks incur a negligible overheads of 2.95% on average. It is because the checking code is executed only once before entering the SIMD loop. As a result, the overhead is amortized. One exception is *JPG-8x8DCT*, which spends 25% total execution time on runtime aliasing checks. The profile data show that its SIMD loops have relatively small trip counts; consequently, the overhead of runtime aliasing checks cannot be fully amortized.

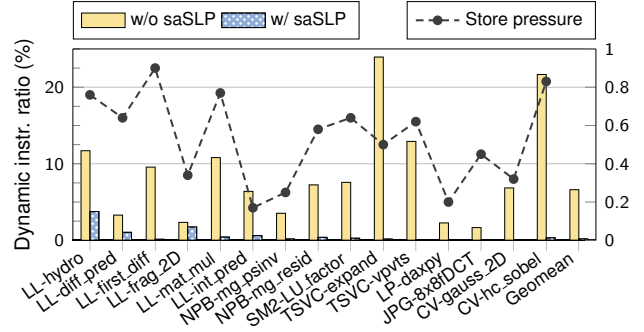
To evaluate the impacts from the automatic unrolling factors made by the guest compiler, we force the guest compiler to compile benchmarks with smaller unrolling factors, which results in no register spilling on the AVX2 host. As shown in Figure 7a, the overall speedup decreases from 1.6X to 1.47X because there is no register spilling to eliminate. However, even in this case, saSLP can still significantly improve the performance by exploiting the AVX2 parallelism.

B. Register Spilling

Recall that the guest compiler automatically unrolls loops according to the 32 available 16-byte NEON registers, resulting in the excessive register pressure on the AVX2 host.



(a) Spill load instruction ratio of NEON to AVX2 translation



(b) Spill store instruction ratio of NEON to AVX2 translation

Figure 8. Register spilling and load/store pressure with AVX2 host

Figure 8 shows the dynamic instruction ratios of register spilling loads (i.e., filling) and stores (i.e., spilling) against the total executed instruction counts, profiled using the Intel Pin tool [34]. On average, saSLP eliminates 67% and 97% of spilling loads and stores respectively by combining 16-byte NEON registers and storing them in 32-byte YMM registers.

For the register spilling loads in Figure 8a, benchmarks like *LL-hydro*, *LL-first_diff*, *NPB-mg_resid*, *TSVC-vpvt*, *LP-daxpy*, *CV-gauss_2D* and *CV-hc_sobel* show a significant reduction in the instruction count for spilling loads when saSLP is applied; for example, in *LL-hydro*, the saSLP bar is only 2.7% of the bar without saSLP. On average, these benchmarks achieve 87% reduction because combining NEON registers in their SIMD loops significantly reduces the maximal number of live ranges that can interfere with each other at the same time. On the other hand, *LL-int_pred* and *JPG-8x8fDCT* do not achieve a significant reduction in spilling loads because all the loads come from NEON registers that are duplicated and packed into YMM registers for their user instructions (e.g., the v4 register in Figure 6b). As these NEON registers are duplicated and fill up the YMM registers rather than being combined with each other, there is no reduction in the number of these live ranges.

For the register spilling stores in Figure 8b, nearly all the benchmarks achieve significant reductions exceeding 70%. The 16-byte NEON live ranges that flow through the SIMD loops for guest state synchronization are combined and stored in the 32-byte YMM registers, and other intra-loop NEON live ranges are also combined. Therefore, the spilling stores in the SIMD loops are eliminated as a consequence of the decrement of interfering NEON live ranges. As shown in Figure 8b, the spilling stores of the SIMD loops in the following benchmarks are fully eliminated when saSLP is applied: *LL-first_diff*, *TSVC-expand*, *TSVC-vpvt*, *LP-daxpy*, *JPG-8x8fDCT*, *CV-gauss_2D* and *CV-hc_sobel*.

The reduction in register spilling can also impact the performance improvement depending on the load/store pressure of the benchmarks. To evaluate the impacts from the memory load/store pressure, we profile the dispatched load/store

micro-ops and memory stall cycles using the hardware performance counters. In Figures 8a and 8b, the dashed lines (right *y*-axis) indicate the ratios of the execution cycles with dispatched load micro-ops (e.g., Skylake port 2/3) over the total execution cycles, and the ratios of the store micro-ops (e.g., Skylake port 4) respectively. The results in Figures 7a and 8, show that the benchmarks with relatively high loads and store pressure *and* significant reduction in register spilling like *LL-hydro* and *LL-first_diff*, achieve speedups exceeding 2X (i.e., the theoretical bound of speedup from improved parallelism). The benchmarks with high store pressure *and* significant reductions in spilling stores, such as *LL-mat_mul*, *TSVC-expand*, *TSVC-vpvt*, also achieve significant speedups ranging from 1.71X to 1.83X.

C. Performance Results with AVX-512

Figure 9a shows the speedup of NEON to AVX-512 translation with saSLP, where four or two NEON instructions are combined to form an AVX-512 or AVX-256 instruction. On average, saSLP achieves a speedup of 2.3X compared to the translation without saSLP. In particular, all benchmarks from the LL, TSVC and CV suites achieve a significant speedup of 2.9X on average. In contrast, *NPB-mg_psinv*, *NPB-mg_resid*, *SM2-LU_factor* and *JPG-8x8fDCT* have an average speedup of 1.5X. As mentioned in Section IV-A, the performance improvement can be impeded by the memory overheads. *NPB-mg_psinv* and *NPB-mg_resid* are benchmarks that suffer from the large working sets and excessive split loads. Moreover, the performance degradation caused by the split memory accesses can be more severe on AVX-512. In this experiment, saSLP combines the guest SIMD instructions to form a long AVX-512 instruction, whose operating size is the same as the 64-byte host cache line size. If the start addresses of the long SIMD memory accesses are not properly aligned (e.g., accesses starting from array $x[i + 1]$ that are shifted by one vector element size), the subsequent long SIMD memory accesses will always cross the 64-byte cache line boundaries and incur a significant penalty [33]. The profile data show that both benchmarks have 37% of split SIMD memory accesses, and

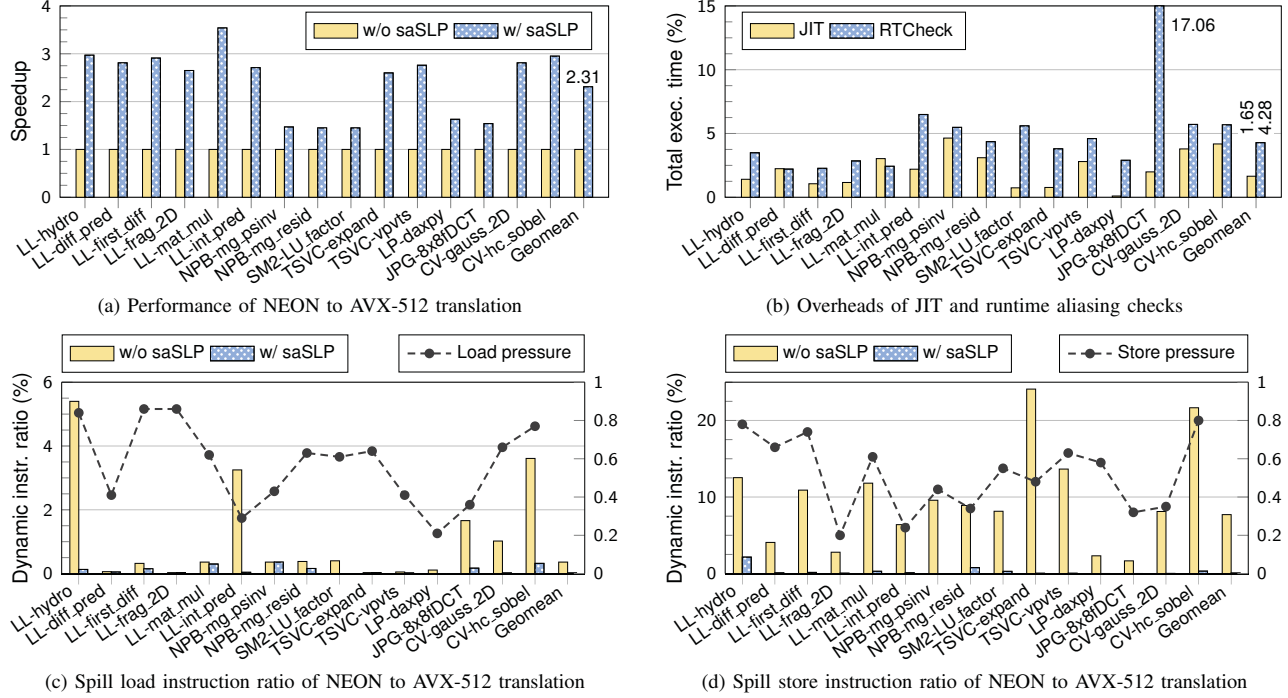


Figure 9. Performance results and register spilling with AVX-512 host

similar phenomenon is also observed in *SM2-LU_factor*. For *JPG-8x8fDCT*, since its SIMD loops process a 8x8 4-byte floating-point pixel block, saSLP only has two 16-byte NEON instructions to combine. Therefore, its performance is similar to that of the translation to AVX2.

As shown in Figure 9b, the overheads of JIT compilation (including optimization) and runtime aliasing checks are similar to those in the translation to AVX2. Because of the amortization of overheads, the JIT compilation and runtime aliasing checks only incur negligible costs of 1.65% and 4.28% of the total execution time on average, respectively.

Figures 9c and 9d show, respectively, the dynamic instruction ratio of register spilling loads and stores with the NEON to AVX-512 translation. On average, saSLP eliminates 95% and 99% of spilling loads and stores respectively. Although AVX-512 supports 32 64-byte ZMM registers, accessing the extra 16 XMM16-31/YMM16-31 sub-registers requires an AVX-512VL extension which is not supported on our host [9]. Therefore, the baseline that translates to XMM width can only use 16 XMM registers, while saSLP can exploit all 32 ZMM registers by combining with the ZMM width. As a consequence, saSLP further reduces register spilling with the extra ZMM registers. For example, the spilling loads caused by the duplicated and packed NEON registers in *LL-int_pred* (Figure 8a) can be eliminated by storing these duplicated NEON registers in the extra ZMM registers, resulting in a 98% reduction in the spilling loads (as shown in Figure 9c). For the spilling store

in Figure 9d, benchmarks like *LL-hydro*, *LL-diff_pred*, *LL-frag2D* and *NPB-mg_psinv* also benefit from the improved SIMD register capacity after the saSLP transformation. On average, there is 82% extra reduction in the spilling stores compared to the AVX2 translation.

Our AVX-512 host machine does not support the hardware counters that count the cycles with dispatched load/store micro-ops to estimate the load/store pressure. Therefore, we estimate the load/store pressure by calculating the ratios of accumulated memory latency (from the event counts and latencies of L1, L2 data cache and DRAM) over the total execution cycles [35]. Figures 9a, 9c and 9d show that, similar to the AVX2 translation, benchmarks with a high load and store pressure *and* significant reduction in register spilling achieve significant speedup. Specifically, *LL-hydro*, *LL-first_diff* and *CV-hc_soble* achieve 2.97X, 2.91X and 2.95X speedup respectively. Besides, benchmarks with high store pressure *and* significant reduction in spilling stores, such as *LL-mat_mul*, *TSVC-expand*, *TSVC-vpvt*s, achieve significant speedups ranging from 2.6X to 3.54X.

To summarize, we evaluated saSLP with the translations from the short SIMD ISA, ARMv8 NEON, to two long SIMD ISAs, x86 AVX2 and AVX-512. The benchmark results show that saSLP can effectively exploit the improved parallelism and the register capacity on the host. Moreover, it can achieve a significant reduction in the register spilling overhead, showing a speedup of 1.6X and 2.3X for the AVX2 and AVX-512 hosts respectively.

V. RELATED WORK

Auto-vectorization approaches are usually classified into two types: traditional loop-based vectorization [36]–[38] and basic-block vectorization [39]–[41]. Among the latter, Superword Level Parallelism (SLP) [39] is the most well-known algorithm. It can combine multiple isomorphic statements into SIMD operations inside a straight-line code. A number of techniques have been developed to improve the quality of SLP vectorization, such as dealing with the presence of control flow [42], padding with non-isomorphic statements [43], partial SIMD parallelism [44]. In this paper, the proposed saSLP extends the implementation of LLVM SLP vectorizer to resolve the asymmetric problems caused by the vectorized and unrolled guest binaries. Our extensions include (1) re-vectorizing short SIMD to long SIMD; (2) runtime aliasing check; and (3) the spill-aware cost model.

Dynamic binary translation (DBT) is a core technology used to execute an application binary of one ISA on a host machine with a different ISA. It operates directly on binaries, allowing it to run legacy or proprietary applications without source code. DBT has been used to achieve many different goals. Dynamic optimization systems [45]–[47] apply runtime binary re-translation to improve the performance; while binary instrumentation systems, e.g., Pin [34] and Valgrind [48], provide features of application analysis and debugging. DBT is also widely used for application migration across ISAs [17], [18], [49]. For example, the cross-ISA DBT systems, such as Digital FX!32 [49], IA-32 EL [18] and Transmeta CMS [50], enabled IA-32 applications to be executed on Alpha, Itanium and VLIW machines in the past.

With the advances in SIMD architectures, several DBT systems that attempt to exploit SIMD extensions have been proposed in the literature. However, most of them have very different objectives and approaches in their designs.

The most closely related works are [14]–[16], which also present vectorization schemes from short SIMD to long SIMD for loops. Hallou et al. [14], [15] designed their work in a same-ISA DBT for the x86 architecture; Hong et al. [16] proposed using a machine-independent IR layer to achieve cross-ISA SIMD transformation. Their approaches conduct SIMD loop transformation based on the loop-based vectorization algorithm. However, they still map one guest SIMD register to one host SIMD register, which cannot eliminate register spilling. In contrast, on the basis of the SLP algorithm, saSLP can combine multiple short guest SIMD registers within an unrolled guest loop and solve the register spilling problem as a result. Our DBT applies loop-based transformation [16] to guest loops which are not unrolled.

Instead of performing vectorization from a short SIMD binary, Pajuelo et al. [51] proposed a binary auto-vectorization framework from scalars to vectors. Their system requires new hardware to execute speculatively vectorized code and falls back to scalar code when an invalid vectorization is

detected by the hardware. Liquid SIMD [52] proposed a general SIMD representation, which can run on any Liquid SIMD-enabled hardware. In contrast, our saSLP transforms short SIMD to long SIMD, and is a pure software solution. Vapor SIMD [53] proposed split vectorization, where offline compilers conduct vectorization of machine-independent parts and runtime JITs resolve the final vectorization based on host SIMD ISA. Different from saSLP, Vapor SIMD is not a DBT.

QEMU [17] emulates a guest SIMD instruction with a sequence of scalar instructions instead of leveraging the host SIMD instructions; that is, it trades the benefits of the host SIMD for portability. Other works [11]–[13] focus on finding efficient translations of SIMD registers and instructions between different SIMD ISAs. In their DBTs, guest SIMD instructions are translated to equivalent host SIMD instructions that operate on the same number of elements even if the host can provide more parallelism. In contrast, our work combines short guest SIMD instructions to form long SIMD instructions so that the host's parallelism is fully utilized.

VI. CONCLUDING REMARKS

The diversity of modern SIMD architectures has become a critical issue when applying DBTs for cross-ISA migration of applications because the number and width of SIMD registers between the guest and host ISA can be very asymmetric. This issue will become more common and severe with the ongoing evolution of SIMD architectures. Translating from longer to shorter SIMD is an easier problem. For example, the type legalization phase in LLVM back-end simply splits each vector register and instruction into a sequence of shorter ones, which preserve the semantic of program. The existing LLVM back-end can be used to handle the transformation from longer to shorter SIMD. Therefore, this paper addresses the more complex optimization problem of leveraging longer SIMD.

We propose a novel cross-ISA DBT technique, called spill-aware SLP (saSLP), which can combine multiple short ARMv8 NEON instructions and registers so as to fully utilize the x86 AVX host's parallelism and register capacity. We also propose the spill-aware cost model and runtime aliasing check to prevent anticipated slowdown and violation of vectorization legality. The evaluation results of saSLP demonstrate that the algorithm can significantly improve the execution efficiency and effectively reduce the register spilling across a range of benchmarks. Moreover, the proposed technique is also applicable to static binary translation, and can be further extended to support more SIMD ISAs since saSLP is built on top of LLVM machine-independent IR layer.

ACKNOWLEDGMENT

This work was supported in part by Ministry of Science and Technology of Taiwan under grant number MOST-105-2218-E-002-017 and MOST-105-2221-E-001-024.

REFERENCES

- [1] J. Lin, “New ways to find (and stream) app content in google search,” 2015. [Online]. Available: <https://search.googleblog.com/2015/11/new-ways-to-find-and-stream-app-content.html>
- [2] J. Steele and N. To, *The Android developer’s cookbook: building applications with the Android SDK*. Pearson Education, 2010.
- [3] S. Ratabouil, *Android NDK: Beginner’s Guide*. Packt Publishing Ltd, 2015.
- [4] G. Portokalidis, P. Homburg, K. Anagnostakis, and H. Bos, “Paranoid android: versatile protection for smartphones,” in *Proceedings of the Annual Computer Security Applications Conference (ACSAC)*. ACM, 2010, pp. 347–356.
- [5] R. Mahmood, N. Esfahani, T. Kacem, N. Mirzaei, S. Malek, and A. Stavrou, “A whitebox approach for automated security testing of android applications on the cloud,” in *Proceedings of the International Workshop on Automation of Software Test (AST)*. IEEE Computer Society, 2012, pp. 22–28.
- [6] S. J. Pennycook, C. J. Hughes, M. Smelyanskiy, and S. A. Jarvis, “Exploring SIMD for molecular dynamics, using Intel Xeon processors and Intel Xeon Phi coprocessors,” in *Proceedings of the International Symposium on Parallel and Distributed Processing (IPDPS)*. IEEE Computer Society, 2013, pp. 1085–1097.
- [7] K. Diefendorff, P. K. Dubey, R. Hochsprung, and H. Scales, “AltiVec extension to powerpc accelerates media processing,” *IEEE Micro*, vol. 20, no. 2, pp. 85–95, 2000.
- [8] J. Zhou and K. A. Ross, “Implementing database operations using SIMD instructions,” in *Proceedings of the ACM International Conference on Management of Data (SIGMOD)*. ACM, 2002, pp. 145–156.
- [9] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer’s Manual*, 2016.
- [10] ARM Ltd., *ARM Cortex-A Series Programmer’s Guide for ARMv8-A*, 2015.
- [11] S. Fu, D. Hong, J. Wu, P. Liu, and W. Hsu, “SIMD code translation in an enhanced HQEMU,” in *Proceedings of the International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE Computer Society, 2015, pp. 507–514.
- [12] J. Li, Q. Zhang, S. Xu, and B. Huang, “Optimizing dynamic binary translation for SIMD instructions,” in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE Computer Society, 2006, pp. 269–280.
- [13] L. Michel, N. Fournel, and F. Pétrot, “Speeding-up SIMD instructions dynamic binary translation in embedded processor simulation,” in *Proceedings of the Design, Automation and Test in Europe (DATE)*. IEEE Computer Society, 2011, pp. 277–280.
- [14] N. Hallou, E. Rohou, and P. Clauss, “Runtime vectorization transformations of binary code,” *International Journal of Parallel Programming*, pp. 1–30, 2016.
- [15] N. Hallou, E. Rohou, P. Clauss, and A. Ketterlin, “Dynamic re-vectorization of binary code,” in *Proceedings of the International Conference on Embedded Computer Systems: Architectures, Modeling, and Simulation (SAMOS)*. IEEE Computer Society, 2015, pp. 228–237.
- [16] D. Hong, S. Fu, Y. Liu, J. Wu, and W. Hsu, “Exploiting longer SIMD lanes in dynamic binary translation,” in *Proceedings of the International Conference on Parallel and Distributed Systems (ICPADS)*. IEEE Computer Society, 2016, pp. 853–860.
- [17] F. Bellard, “Qemu, a fast and portable dynamic translator,” in *Proceedings of the USENIX Annual Technical Conference (USENIX)*. USENIX, 2005, pp. 41–46.
- [18] L. Baraz, T. Devor, O. Etzion, S. Goldenberg, A. Skaletsky, Y. Wang, and Y. Zemach, “IA-32 execution layer: a two-phase dynamic translator designed to support IA-32 applications on itanium-based systems,” in *Proceedings of the Annual International Symposium on Microarchitecture (MICRO)*. ACM/IEEE Computer Society, 2003, pp. 191–204.
- [19] B. Dolan-Gavitt, J. Hodosh, P. Hulin, T. Leek, and R. Whelan, “Repeatable reverse engineering with PANDA,” in *Proceedings of the Program Protection and Reverse Engineering Workshop (PPREW@ACSAC)*. ACM, 2015, pp. 4:1–4:11.
- [20] J. W. Davidson and S. Jinturkar, “Improving instruction-level parallelism by loop unrolling and dynamic memory disambiguation,” in *Proceedings of the Annual International Symposium on Microarchitecture (MICRO)*. ACM/IEEE, 1995, pp. 125–132.
- [21] —, “An aggressive approach to loop unrolling,” Department of Computer Science, University of Virginia, Charlottesville, Tech. Rep., 1995.
- [22] Y. Ma, “Register pressure guided loop optimization,” Ph.D. dissertation, Michigan Technological University, Houghton, MI, USA, 2007.
- [23] R. Ira, N. Dorit, and Z. Ayal, “Loop-aware SLP in GCC,” in *Proceedings of the GCC Developers Summit*, 2007, pp. 131–142.
- [24] C. Lattner and V. S. Adve, “LLVM: A compilation framework for lifelong program analysis & transformation,” in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE Computer Society, 2004, pp. 75–88.
- [25] A. W. Appel and M. Ginsburg, *Modern Compiler Implementation in C*. New York, NY, USA: Cambridge University Press, 2004.
- [26] F. McMahon, “The livermore fortran kernels: A computer test of the numerical performance range,” Lawrence Livermore National Lab, Tech. Rep., 1986.
- [27] D. H. Bailey, E. Barszcz, J. T. Barton, D. S. Browning, R. L. Carter, L. Dagum, R. A. Fatoohi, P. O. Frederickson, T. A. Lasinski, R. Schreiber, H. D. Simon, V. Venkatakrishnan, and S. Weeratunga, “The NAS parallel benchmarks,” *International Journal of High Performance Computing Applications (IJHPCA)*, vol. 5, no. 3, pp. 63–73, 1991.

- [28] R. Pozo and B. Miller, “SciMark 2.0 benchmark,” 2004. [Online]. Available: <http://math.nist.gov/scimark2>
- [29] S. Maleki, Y. Gao, M. J. Garzarán, T. Wong, and D. A. Padua, “An evaluation of vectorizing compilers,” in *Proceedings of the International Conference on Parallel Architectures and Compilation Techniques (PACT)*. IEEE Computer Society, 2011, pp. 372–382.
- [30] J. J. Dongarra, C. B. Moler, J. R. Bunch, and G. W. Stewart, *LINPACK users’ guide*. SIAM, 1979.
- [31] Independent JPEG Group, “IJG JPEG library,” 2016. [Online]. Available: <http://www.ijg.org>
- [32] Itseez, “Open source computer vision library,” 2017. [Online]. Available: <http://opencv.org>
- [33] Intel Corporation, *Intel 64 and IA-32 Architectures Optimization Reference Manual*, 2016.
- [34] C. Luk, R. S. Cohn, R. Muth, H. Patil, A. Klauser, P. G. Lowney, S. Wallace, V. J. Reddi, and K. M. Hazelwood, “Pin: building customized program analysis tools with dynamic instrumentation,” in *Proceedings of the Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2005, pp. 190–200.
- [35] J. Jeffers, J. Reinders, and A. Sodani, *Intel Xeon Phi Processor High Performance Programming: Knights Landing Edition*. Elsevier Science, 2016.
- [36] N. Sreraman and R. Govindarajan, “A vectorizing compiler for multimedia extensions,” *International Journal of Parallel Programming*, vol. 28, no. 4, pp. 363–400, 2000.
- [37] A. J. C. Bik, M. Girkar, P. M. Grey, and X. Tian, “Automatic intra-register vectorization for the Intel architecture,” *International Journal of Parallel Programming*, vol. 30, no. 2, pp. 65–98, 2002.
- [38] D. Naishlos, “Autovectorization in GCC,” in *Proceedings of the GCC Developers Summit*, 2004, pp. 105–118.
- [39] S. Larsen and S. P. Amarasinghe, “Exploiting superword level parallelism with multimedia instruction sets,” in *Proceedings of the Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2000, pp. 145–156.
- [40] R. Leupers and S. Bashford, “Graph-based code selection techniques for embedded processors,” *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 5, no. 4, pp. 794–814, 2000.
- [41] S. Kral, F. Franchetti, J. Lorenz, and C. W. Ueberhuber, “SIMD vectorization of straight line FFT code,” in *Proceedings of the International European Conference on Parallel and Distributed Computing (Euro-Par)*. Springer, 2003, pp. 251–260.
- [42] J. Shin, M. W. Hall, and J. Chame, “Superword-level parallelism in the presence of control flow,” in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE Computer Society, 2005, pp. 165–175.
- [43] V. Porpodas, A. Magni, and T. M. Jones, “PSLP: padded SLP automatic vectorization,” in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE Computer Society, 2015, pp. 190–201.
- [44] H. Zhou and J. Xue, “A compiler approach for exploiting partial SIMD parallelism,” *ACM Transactions on Architecture and Code Optimization (TACO)*, vol. 13, no. 1, pp. 11:1–11:26, 2016.
- [45] V. Bala, E. Duesterwald, and S. Banerjia, “Dynamo: a transparent dynamic optimization system,” in *Proceedings of the Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2000, pp. 1–12.
- [46] J. Lu, H. Chen, P. Yew, and W. Hsu, “Design and implementation of a lightweight dynamic optimization system,” *Journal of Instruction-Level Parallelism*, vol. 6, 2004.
- [47] C. Wang, S. Hu, H. Kim, S. R. Nair, M. B. Jr., Z. Ying, and Y. Wu, “Stardbt: An efficient multi-platform dynamic binary translation system,” in *Proceedings of the Asia-Pacific Computer Systems Architecture Conference*, ser. Lecture Notes in Computer Science. Springer, 2007, pp. 4–15.
- [48] N. Nethercote and J. Seward, “Valgrind: a framework for heavyweight dynamic binary instrumentation,” in *Proceedings of the Conference on Programming Language Design and Implementation (PLDI)*. ACM, 2007, pp. 89–100.
- [49] A. Chernoff, M. A. Herdeg, R. J. Hookway, C. Reeve, N. Rubin, T. Tye, S. B. Yadavalli, and J. Yates, “FX!32 a profile-directed binary translator,” *IEEE Micro*, vol. 18, no. 2, pp. 56–64, 1998.
- [50] J. C. Dehnert, B. Grant, J. P. Banning, R. Johnson, T. Kistler, A. Klaiber, and J. Mattson, “The transmeta code morphing - software: Using speculation, recovery, and adaptive retranslation to address real-life challenges,” in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE Computer Society, 2003, pp. 15–24.
- [51] A. Pajuelo, A. González, and M. Valero, “Speculative dynamic vectorization,” in *Proceedings of the International Symposium on Computer Architecture (ISCA)*. IEEE Computer Society, 2002, pp. 271–280.
- [52] N. Clark, A. Hormati, S. Yehia, S. A. Mahlke, and K. Flautner, “Liquid SIMD: abstracting SIMD hardware using lightweight dynamic mapping,” in *Proceedings of the International Conference on High-Performance Computer Architecture (HPCA)*. IEEE Computer Society, 2007, pp. 216–227.
- [53] D. Nuzman, S. Dyshel, E. Rohou, I. Rosen, K. Williams, D. Yuste, A. Cohen, and A. Zaks, “Vapor SIMD: auto-vectorize once, run everywhere,” in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE Computer Society, 2011, pp. 151–160.